AI-200 Hands-On Practice - 7-Day Plan

Goal: Rebuild Azure AI cloud solution knowledge through one project instead of re-reading course modules.
Project: Containerized FastAPI document API -> Azure Functions -> Service Bus -> Cosmos DB -> PostgreSQL + pgvector -> Redis, plus security and monitoring.

Day	Topic	Key Azure services
1	Containerized API + deploy (finish)	Docker, Container Apps
2	Blob storage + Functions	Blob Storage, Azure Functions
3	Reliable messaging	Event Grid, Service Bus, worker
4	Data layer	Cosmos DB, PostgreSQL + pgvector
5	Caching + security	Redis, Key Vault, Entra ID, RBAC
6	Monitoring + troubleshooting	Application Insights, KQL
7	Rebuild from memory	Full integrated pipeline

Day 1 - Containerized API + Azure deploy (finish)

Deliverable

- Fix the containerapp extension: run az upgrade.
- Run: az extension remove --name containerapp
- Run: az extension add --name containerapp --upgrade --allow-preview true
- Restart terminal, re-login, restore RESOURCE_GROUP and CONTAINER_APP variables.
- Verify: az containerapp show returns app details (no InvalidApiVersionParameter).
- Run: curl https:///health and confirm it returns 200.
- Test POST /documents and GET /documents/{id} against Azure.
- Inspect container logs and run az containerapp revision list.

Concepts to retain

- Image (read-only template) vs container (running instance).
- EXPOSE 8000 vs --target-port 8000.
- Uvicorn must bind 0.0.0.0 inside a container.
- Why in-memory doc store is not production-ready (restart clears it).
- How az containerapp up builds the local Dockerfile.

Day 2 - Blob Storage + Azure Functions

Deliverable

- Create a storage account and containers (e.g. uploads / processed).
- Deploy a Python Azure Function App (v2 model) with a Blob-triggered function; set Path: uploads/{name}.
- On blob create: generate document_id, write a status record (uploaded), return quickly (no heavy work).
- Define function.json bindings clearly.
- Run the function locally with Azure Functions Core Tools, then deploy via Azure CLI / VS Code.
- Upload a file to the uploads container and verify the function log shows the blob was read.

Concepts to retain

- Azure Functions triggers and bindings.
- Blob trigger starts on new/updated blob; blob contents are passed as input.
- Python v1 (function.json) vs v2 (Python decorators) programming models.

Day 3 - Event Grid + Service Bus

Deliverable

- Create a Service Bus namespace and a queue (e.g. document-processing).
- The Function sends a message (document_id + blob URI) to the queue using azure-servicebus [web:92].
- Create a Python worker that receives messages, downloads the blob, and updates status to processing/completed.
- Make the consumer idempotent: the same message must not process twice (dedupe check).
- Break it: crash before complete_message, deliver twice, send bad JSON; verify retry and dead-letter behavior.
- Add a note explaining when to use Event Grid vs Service Bus.

Concepts to retain

- Event Grid = event notification; Service Bus = durable reliable messaging.
- complete_message vs abandon / retry; dead-letter queue.
- Idempotency and poison-message handling.

Day 4 - Cosmos DB + PostgreSQL pgvector

Deliverable

- Cosmos DB (NoSQL): store document metadata, processing status, chat history. Choose a partition key, point reads, query, use TTL.
- PostgreSQL flexible server with pgvector: store document chunks, embeddings, source refs.
- Implement chunking, embedding generation, vector insert, similarity search with metadata filtering.
- Ask 5 questions about uploaded docs; verify retrieved source/page is correct.
- Compare semantic vs keyword search; record results.

Concepts to retain

- Choosing a Cosmos DB partition key; RU consumption.
- pgvector similarity search and where it fits vs Cosmos DB vector search.

Day 5 - Redis caching + security

Deliverable

- Provision Azure Managed Redis; connect from Python with azure-redis and Entra ID [web:93].
- Cache repeated question results with key chat:{user_id}:{question_hash} [web:102].
- Test cache hit/miss, expiry, and invalidate after a document changes.
- Replace connection strings where possible with managed identity / Azure Key Vault.
- Create a runtime identity with least-privilege RBAC roles (read storage, publish/consume messages, read secrets, access DBs).

- Remove one role and verify the app fails loudly (clear error, not silent wrong output).

Concepts to retain

- Managed identity vs secrets; Key Vault; least-privilege RBAC.
- Cache invalidation strategy.

Day 6 - Monitoring + troubleshooting

Deliverable

- Enable Application Insights on the function and the container app.
- Instrument a correlation ID through API -> Function -> Service Bus -> Worker -> DB.
- Track: request duration, function failures, queue depth, dead-letter count, worker failures, DB latency, Redis hit ratio, AI token usage.
- Write at least 5 KQL queries (failed requests, slowest API, exceptions, by operation, by correlationId).
- Run deliberate breakage: deny DB access, stop consumer, deploy a bad revision; record symptom -> metric -> log query -> fix. Azure Container Apps provides console and system logs [web:80].

Concepts to retain

- Distributed observability with OpenTelemetry.
- Console vs system log types for troubleshooting.

Day 7 - Rebuild from memory + readiness

Deliverable

- Without copying old code, rebuild: containerized API -> blob trigger -> Service Bus -> worker -> status record -> vector search -> cached result -> find one failure via KQL.
- Answer AI-200 checkpoint questions from memory: Functions vs Container Apps, Event Grid vs Service Bus, Cosmos partition key, pgvector latency, cache invalidation, managed identity, dead-letter, rollback, cross-service tracing.
- Write README + architecture diagram + troubleshooting runbook.
- Clean up: az group delete --name "$RESOURCE_GROUP" --yes --no-wait

Concepts to retain

- Full AI-200 coverage: compute, containers, Functions, messaging/eventing, Azure data services, vector DB, security, observability, troubleshooting.

Daily study method (60-90 min)

- 10 min: write what you remember before opening docs.
- 40-60 min: implement one feature.
- 10 min: break the system deliberately, then troubleshoot.
- 10 min: write three lessons learned + flashcards from mistakes.